

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ
по дисциплине «Методы тестирования программного
обеспечения»
на тему «Инспекция кода»

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Курочкин М. А. _____

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Методы ручного тестирования	5
2.1 Инспекция кода	5
2.1.1 Контрольные списки возможных ошибок для инспекций	6
2.2 Сквозные просмотры	11
2.3 Проверка за столом	12
2.4 Рецензирование	12
3 Описание тестируемой программы	14
4 Описание проведенной инспекции кода	24
4.1 Протокол заседания	24
4.2 Полученные рекомендации	27
4.3 Исправленный код	28
Заключение	38

Введение

Ручное тестирование программ не предполагает использование компьютера и осуществляется непосредственно человеком. Оно зарекомендовало себя как достаточно эффективный метод обнаружения ошибок, и его разновидности следует задействовать в любом программном проекте.

Двумя основными методами ручного тестирования являются инспекция кода и сквозной просмотр. Они могут применяться практически на любой стадии разработки программного обеспечения, как к отдельным готовым модулям или блокам, так и к приложению в целом.

Инспекция и сквозной просмотр включают в себе чтение или визуальную проверку программного кода группой лиц. Завершающим этапом является обмен мнениями между участниками проверки на специальном заседании. Цель такого заседания – нахождение ошибок, а не их устранение.

Как правило, ручными методами тестирования удастся обнаруживать от 30 до 70% ошибок, связанных с проектированием и кодированием логики типичных программ. В тоже время они не эффективны при выявлении ошибок высокоуровневого проектирования, например ошибок, допущенных на стадии анализа требований.

1 Постановка задачи

Цель работы: провести инспекцию кода, выступая в роли разработчика программы.

Задачи:

- изучить методы ручного тестирования;
- подготовить программу для тестирования;
- провести инспекцию кода программы;
- проанализировать результаты тестирования;
- исправить программу в соответствии с рекомендациями специалиста по тестированию.

2 Методы ручного тестирования

2.1 Инспекция кода

Инспекция кода – это набор процедур и методик обнаружения ошибок путем анализа (чтения) кода группой специалистов.

Обычно в состав группы инспектирования кода входят четыре человека, один из которых играет роль координатора. Координатор должен быть квалифицированным программистом, но не автором тестируемой программы, детальное знание которой от него не требуется. В обязанности координатора входит следующее:

- раздача материалов участникам заседания инспекционной группы и составление плана его проведения;
- проведение заседания;
- запись всех обнаруженных ошибок;
- последующая проверка того, что обнаруженные ошибки устранены.

Также в состав участников группы входят программист, проектировщик программы (если это не сам программист) и специалист по тестированию. Специалист по тестированию должен хорошо ориентироваться в методах тестирования ПО и знать наиболее распространенные типы ошибок.

Порядок проведения заседаний по инспектированию кода

За несколько дней до заседания координатор раздает листинг программы и спецификацию проекта всем участникам группы для того, чтобы к моменту проведения заседания участники успели ознакомиться с необходимыми материалами. Инспекционное заседание проводится в соответствии с приведенным ниже порядком.

1. Программист рассказывает присутствующим о логике работы программы, подробно останавливаясь на каждой инструкции. В это время другие участники заседания должны задавать ему вопросы, которые способствуют выявлению возможных ошибок. Вполне вероятно, что по ходу дела часть ошибок обнаружит сам программист, а не остальные участники группы.
2. Участники заседания анализируют код программы, сверяясь с составленными на основе имеющегося опыта контрольными списками наиболее распространенных ошибок.

По окончании заседания программисту передастся список найденных ошибок. Если этот список достаточно длинный или устранение ошибок требует внесения существенных изменений в программу, координатор может принять решение о повторной инспекции кода после того, как ошибки будут исправлены. Список анализируется и после разбиения ошибок на категории дополняет имеющийся контрольный список, что позволяет повысить эффективность будущих инспекций кода.

Процесс инспектирования кода нацелен не на устранение ошибок, а на их обнаружение. Но если выявленная проблема оказалась не слишком серьезной, то группа может счесть целесообразным, чтобы два-три человека, включая разработчика программы, сформулировали предложения по внесению в проект необходимых изменений.

Время и место проведения инспекции должны быть спланированы так, чтобы никакие внешние факторы не могли мешать работе заседания. Оптимальная длительность таких заседаний составляет от полутора до двух часов. С увеличением их длительности продуктивность может резко падать. В большинстве случаев инспекция кода осуществляется со средней скоростью 150 строк в час.

2.1.1 Контрольные списки возможных ошибок для инспекций

Важной частью процесса инспектирования кода является использование контрольных списков, облегчающих обнаружение наиболее распространенных ошибок в программах.

Ошибки обращения к данным

1. Существуют ли обращения к неинициализированным переменным или переменным, которым не были присвоены значения?
2. Не выходят ли значения индексов за установленные пределы при обращении к массиву?
3. Принимают ли индексы исключительно целочисленные значения при любых обращениях к массиву?
4. Во всех ли случаях заблаговременно выделяется память, к которой выполняется обращение с помощью указателя или ссылки?
5. Если область памяти имеет несколько псевдонимов с различными атрибутами, то имеют ли данные, хранящиеся в этой области, корректные атрибуты при обращении к ним по любому из имен?
6. Отличаются ли типы и атрибуты переменных от тех, которые подразумевает компилятор?

7. Возникают ли явные или неявные проблемы адресации, если размер распределяемых блоков памяти в используемом компьютере меньше размера адресуемых блоков?
8. Если используются указатели или ссылочные переменные, то совпадают ли атрибуты адресуемой памяти с теми, которые ожидаются компилятором?
9. Если к структуре данных имеются обращения из нескольких процедур или подпрограмм, то идентичны ли определения этой структуры в каждой из процедур?
10. Не происходит ли выход на один символ за границы строки при ее индексации?
11. Все ли правила наследования соблюдены при реализации классов?

Ошибки описания данных

1. Все ли переменные объявлены непосредственным образом?
2. Если в объявлении переменной указаны не все атрибуты, то достаточно ли понятно, какие атрибуты будут использованы по умолчанию?
3. Правильно ли инициализируются переменные, которые получают начальные значения в декларативных инструкциях?
4. Правильно ли определяются размер и тип каждой переменной?
5. Соплассуетса ли инициализация переменной с ее типом памяти?
6. Есть ли переменные со сходными именами?

Ошибки вычислений

1. Имеются ли вычисления, в которых участвуют переменные несоответствующего типа?
2. Имеются ли вычисления, в которых смешиваются данные разных типов?
3. Имеются ли вычисления, в которых используются переменные одного типа, но различной длины?
4. Имеет ли тип переменной, находящейся в левой части оператора присваивания, меньшую длину, чем тип переменной или результат вычисления выражения в правой части?

5. Возможны ли переполнение или потеря точности при вычислении выражения?
6. Возможно ли обращение делимого в нуль при выполнении операции деления?
7. Если машинным представлением значений переменных является двоичное, то может ли результирующая неточность иметь какие-либо последствия?
8. Может ли переменная выходить за пределы допустимого по логике задачи диапазона значений?
9. Верны ли предположения относительно порядка вычисления выражений и приоритета операций в выражениях, включающих несколько операций?
10. Встречается ли неправильное использование арифметики целых чисел, особенно при делении?

Ошибки сравнения

1. Сравниваются ли в программе переменные различных типов?
2. Сравниваются ли в программе переменные различной длины?
3. Правильно ли используются операторы сравнения?
4. Каждое ли булево выражение имеет тот смысл, который в него вкладывался?
5. Являются ли операнды булевых выражений булевыми?
6. Сравниваются ли в программе между собой числа с плавающей точкой или дробные числа, представленные компьютером в двоичной форме?
7. Верны ли предположения относительно порядка вычислений и приоритета операций в выражениях, содержащих более одной булевой операции?
8. Зависит ли работа программы от способа разбора булевых выражений компилятором?

Ошибки в управляющей логике

1. Если в программе имеется множественное ветвление, то может ли значение индекса когда-либо превысить число возможных переходов?
2. Сможет ли каждый из циклов в конечном итоге завершиться?
3. Сможет ли программа, модуль или подпрограмма в конечном итоге завершиться?
4. Возможно ли, что в силу входных условий выполнение цикла никогда не начнется? Если да, то является ли это просчетом?
5. Каковы могут быть последствия “погружения в цикл”, если цикл управляется как числом итераций, так и булевыми условиями?
6. Имеются ли ошибки диапазона, из-за которых цикл выполняется на одну итерацию больше или меньше, чем требуется?
7. Если в языке существуют понятия групп инструкций или блоков кода (например, $\{\dots\}$), то находится ли для каждой группы своя запись `while` и соответствуют ли записи `do` своим группам?
8. Учтены ли все возможные условия в точках ветвления?

Ошибки программных интерфейсов

1. Совпадает ли число формальных параметров, ожидаемых данным модулем, с числом аргументов, передаваемых ему каждым из вызывающих модулей? Правильен ли порядок следования аргументов?
2. Совпадают ли атрибуты параметров с атрибутами соответствующих аргументов?
3. Совпадают ли единицы измерения параметров и соответствующих аргументов?
4. Совпадает ли число аргументов, передаваемых данным модулем другому модулю, с числом параметров, ожидаемых вызываемым модулем?
5. Совпадают ли атрибуты аргументов, передаваемых другому модулю, с атрибутами соответствующих параметров этого модуля?
6. Совпадают ли единицы измерения аргументов, переданных другому модулю, и параметров этого модуля?
7. Если вызываются встроенные функции, то правильно ли использованы количество, атрибуты и порядок следования аргументов?

8. Если модуль или класс имеет несколько точек входа, то имеются ли в нем обращения к параметру, не связанному с текущей точкой входа?
9. Пытается ли подпрограмма изменить значение параметра, который должен использоваться только как входная величина?
10. Если имеются глобальные переменные, то совпадают ли их определения и атрибуты во всех модулях, которые к ним обращаются?
11. Передаются ли константы в качестве аргументов?

Ошибки ввода-вывода

1. Правильно ли заданы атрибуты в объявлениях файлов?
2. Правильно ли заданы атрибуты в инструкциях OPEN для файлов?
3. Соплассуетса ли спецификация формата с информацией, указанной в инструкциях ввода-вывода?
4. Хватает ли свободной памяти для хранения файла, который будет считываться программой?
5. Все ли файлы открываются перед обращением к ним?
6. Все ли файлы закрываются после обращения к ним?
7. Корректно ли обнаруживаются и обрабатываются признаки конца файла?
8. Корректно ли обрабатываются ошибки ввода-вывода?
9. Замечены ли орфографические или грамматические ошибки в тексте, выводимом программой на печать или на экран?
10. Обрабатывает ли программа ошибки типа “Файл не найден”?

Другие ошибки

1. Если компилятор генерирует список перекрестных ссылок, тщательно изучите его для выявления переменных, на которые ссылки вообще отсутствуют или имеется всего лишь одна ссылка.
2. Если компилятор генерирует список атрибутов, проверьте атрибуты каждой переменной для выявления возможных некорректных атрибутов, присвоенных по умолчанию.

3. Если программа успешно транслируется, но компилятор выдает одно или несколько предупреждений или информационных сообщений, внимательно изучите каждое из них.
4. Достаточно ли надежна программа или модуль? Иными словами, проверяют ли они корректность входных данных?
5. Не пропущена ли в программе какая-нибудь функция?

2.2 Сквозные просмотры

Сквозной просмотр, как и инспекция, представляет собой совокупность процедур и методов обнаружения ошибок в коде путем его просмотра участниками группы тестирования.

Подобно инспекции, сквозной просмотр проводится в форме непрерывного заседания, длящегося не более двух часов. Число участников группы, выполняющей сквозной просмотр, составляет 3-5 человек. Один из них выполняет функции, аналогичные функциям координатора в группе инспектирования. Еще один человек играет роль секретаря и записывает все найденные ошибки, а третий участник — тестировщик. Мнения о том, кто должен входить в группу и стоит ли ее расширять до пяти человек, расходятся. Конечно, одним из этих людей должен быть программист, написавший программу. К числу других возможных кандидатур относятся: высококвалифицированный программист, эксперт по используемому языку программирования, начинающий программист, человек, который будет обслуживать программу, участник какого-либо другого проекта и кто-нибудь из той же группы программистов, что и автор программы.

Процедура сквозного просмотра начинается точно так же, как и процедура инспекции: за несколько дней до заседания необходимые материалы раздают участникам, чтобы они могли заблаговременно ознакомиться с проверяемым программным обеспечением. Однако работа самого заседания организуется иначе. Тот, кто был назначен тестировщиком, приходит на совещание с предварительно записанными на листах бумаги тестами — представительными наборами входных и ожидаемых выходных данных для программы или модуля. Во время заседания участники “прогоняют в уме” каждый тест, то есть подвергают тестовые данные обработке вручную в соответствии с логикой программы. Состояние программы отслеживается на бумаге или на доске.

Сквозные просмотры должны проходить так же, как и описанный ранее процесс инспектирования.

2.3 Проверка за столом

Третьим методом ручного обнаружения ошибок является *"проверка за столом"*. Она может рассматриваться как инспекция или сквозной просмотр кода, выполняемые одним человеком, который вычитывает код программы, проверяет его, руководствуясь контрольным списком ошибок, и прогоняет через логику программы тестовые данные.

Для большинства людей проверка за столом является относительно непродуктивной. Это объясняется прежде всего тем, что такая проверка представляет собой полностью неупорядоченный процесс. Вторая, более важная причина заключается в том, что проверка за столом противоречит второму принципу тестирования, согласно которому тестирование программистом собственных программ оказывается неэффективным. Следовательно, оптимальный вариант состоит в том, чтобы такую проверку выполнял человек, не являющимся автором программы. Например, два программиста могут обмениваться программами для взаимной проверки, но даже в этом случае такая проверка менее эффективна, чем сквозные просмотры или инспекции.

2.4 Рецензирование

Рецензирование – это процедура анонимной оценки общих характеристик качества, обслуживаемости, расширяемости, удобства использования и ясности программного обеспечения. Цель данного метода – предоставить программисту возможность получить стороннюю оценку результатов своего труда.

Общее руководство процессом осуществляет администратор, выбираемый из числа программистов. В свою очередь, администратор отбирает в группу рецензентов от 6 до 20 участников. Предполагается, что все участники специализируются в одной области. Каждый из участников предоставляет для рецензирования две своих программы, одну из которых он считает наилучшей, а вторую – наихудшей по качеству.

Когда будут собраны все программы, их распределяют случайным образом между участниками. Каждому участнику дают для рецензирования четыре программы. Две из них относятся к категории “наилучших”, а две — к категории “наихудших” программ, но рецензенту не сообщают, какой именно является каждая из них. Любой участник тратит на просмотр одной программы 30 минут и заполняет ее оценочную анкету. После просмотра всех четырех программ рецензент оценивает их относительное качество.

Анкета предлагает оценивать программы по десятибалльной шкале, отвечая, например, на следующие вопросы.

1. Легко ли было разобраться в программе?
2. Легко ли вам было бы модифицировать данную программу?

3. Испытывали бы вы чувство удовлетворения, написав такую программу?
4. Хорошо ли различимы и достаточно ли разумны используемые в программе концепции высокоуровневого проектирования?

Рецензента также просят предоставить свои замечания к программе и дать рекомендации по ее улучшению.

После просмотра всех программ каждому участнику передают анкеты с оценками двух его программ. Кроме того, участники получают статистическую сводку, отражающую общие и детализированные данные о рейтинге их собственных программ среди всего набора, а также анализ того, насколько оценки, данные участником чужим программам, близки к оценкам тех же программ со стороны других рецензентов.

3 Описание тестируемой программы

Название программы

Вычисление кратчайшего пути в связном ациклическом неориентированном графе методом Беллмана-Форда.

Входные данные

- `int vertex` – количество вершин в графе;
- `int in` – номер действия пользовательского интерфейса;
- `int start, finish` – индексы начальной и конечной вершины пути на графе.

Ограничения

Все параметры – целые числа:

- $vertex \in [2, 100]$;
- $in \in [0, 2]$;
- $start \in [1, vertex]$;
- $finish \in [1, vertex]$.

Спецификация

В таблице 1 приведена спецификация программы: выходные данные и реакция программы на граничные условия входных данных.

Исходный код программы

В листингах 1, 2 и 3 приведен исходный код тестируемой программы.

Таблица 1: Таблица входных и выходных данных программы

Входные данные				Выходные дан- ные	Реакция про- граммы
vertex	in	start	finish		
$\notin [2, 100]$	-	-	-	Сообщение: «Please, enter the integer from 2 to 100»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение коли- чества вершин в графе.
$\in [2, 100]$	-	-	-	Степени вершин графа, матрица весов графа и меню действий.	Ожидание ввода пользователя.
$\in [2, 100]$	$\notin [0, 2]$	-	-	Сообщение: «Please, enter the integer number from 0 to 2»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение действия меню.
$\in [2, 100]$	2	-	-	Сообщение: «Enter the number of vertex you want to start from (from 1 to vertex):»	Программа ожи- дает ввода номера стартовой верши- ны пути.
$\in [2, 100]$	2	$\notin [1, \text{vertex}]$	-	Сообщение: «Input error. Please, enter the value from 0 to vertex»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение.
$\in [2, 100]$	2	$\in [1, \text{vertex}]$	-	Сообщение: «Enter the number of vertex you want to finish (from 1 to vertex):»	Программа ожи- дает ввода номера конечной верши- ны пути.
$\in [2, 100]$	2	$\in [1, \text{vertex}]$	$\notin [1, \text{vertex}]$	Сообщение: «Input error. Please, enter the value from 0 to vertex»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение.
$\in [2, 100]$	2	$\in [1, \text{vertex}]$	$\in [1, \text{vertex}]$	Сообщение с дли- ной минимального пути, меню дей- ствий	Ожидание ввода пользователя.
$\in [2, 100]$	1	-	-	Сообщение: «Enter the number of vertices: »	Программа ожи- дает ввод количе- ства вершин для нового графа
$\in [2, 100]$	0	-	-	-	Завершение рабо- ты.

```

1  #pragma once
2  #include <iostream>
3  #include <vector>
4  #include <queue>
5  #include <map>
6  #include <set>
7  #include <unordered_set>
8  #include <stack>
9  #include <random>
10 #include <algorithm>
11 #include <numeric>
12 #include <limits>
13 #include <ctime>
14 #include <iomanip>
15 #include <string>
16 #include <fstream>
17 #include <sstream>
18
19 class Graph
20 {
21 private:
22     int numOfVertex;
23     std::vector<int> degrees;
24     std::vector<std::vector<int>> adjacencyMatrix;
25     std::vector<std::vector<int>> weightMatrix;
26
27     double randomDouble() {
28         static std::random_device rd;
29         static std::mt19937 gen(rd());
30         static std::uniform_real_distribution<> dis(0.0, 1.0);
31         return dis(gen);
32     }
33 public:
34     Graph() : numOfVertex(0) {}
35     Graph(int vertex);
36
37     void generateDegrees(double lambda);
38     void createAdjMatrix();
39     void generateWeightMatrix(double lambda);
40
41     bool BellmanFord(int start, int finish, std::vector<int>&
42         distances, std::vector<int>& path, int& iterations);
43
44     int getNumOfVertex() const;
45     const std::vector<int> getDegrees();
46     const std::vector<std::vector<int>> getAdjMatrix() const;
47     const std::vector<std::vector<int>> getWeightMatrix() const;
48 };

```

Листинг 1: Исходный код Graph.h

```

1  #include "Graph.h"

```



```

2
3 Graph::Graph(int vertex) : numOfVertex(vertex)
4 {
5     adjacencyMatrix.resize(numOfVertex, std::vector<int>(
6         numOfVertex, 0));
7     weightMatrix.resize(numOfVertex, std::vector<int>(numOfVertex
8         , 0));
9     degrees.resize(numOfVertex, 0);
10 }
11
12 int Graph::getNumOfVertex() const { return numOfVertex; }
13
14 const std::vector<int> Graph::getDegrees() { return degrees; }
15
16 const std::vector<std::vector<int>> Graph::getAdjMatrix() const {
17     return adjacencyMatrix; }
18
19 const std::vector<std::vector<int>> Graph::getWeightMatrix()
20     const { return weightMatrix; }
21
22 void Graph::generateDegrees(double lambda) {
23     degrees.clear();
24     degrees.resize(numOfVertex, 0);
25
26     for (int i = 0; i < numOfVertex; ++i) {
27         double r_i = randomDouble();
28         r_i = std::max(r_i, 1e-10);
29
30         double x_i = -log(r_i) / lambda;
31
32         int degree = static_cast<int>(round(x_i));
33
34         int max_possible_degree = numOfVertex - i - 1;
35         if (max_possible_degree > 0) {
36             degree = std::min(degree, max_possible_degree);
37         }
38         else {
39             degree = 0;
40         }
41
42         degrees[i] = std::max(degree, 0);
43     }
44
45     std::sort(degrees.begin(), degrees.end(), std::greater<int>());
46
47     bool isValid = false;
48     while (!isValid) {
49         isValid = true;
50
51         for (int i = 0; i < numOfVertex; ++i) {
52             if (degrees[i] > numOfVertex - i - 1) {

```

```

49         isValid = false;
50         break;
51     }
52 }
53
54 if (!isValid) {
55     for (int i = 0; i < numOfVertex; ++i) {
56         double r_i = randomDouble();
57         r_i = std::max(r_i, 1e-10);
58         double x_i = -log(r_i) / lambda;
59         int degree = static_cast<int>(round(x_i));
60         int max_possible_degree = numOfVertex - i - 1;
61         degrees[i] = std::min(std::max(degree, 0),
62                               max_possible_degree);
63     }
64     std::sort(degrees.begin(), degrees.end(), std::
65               greater<int>());
66 }
67
68 if (numOfVertex == 2) {
69     degrees[0] = 1;
70     degrees[1] = 0;
71 }
72 degrees[numOfVertex - 1] = 0;
73 }
74
75 void Graph::createAdjMatrix()
76 {
77     adjacencyMatrix.assign(numOfVertex, std::vector<int>(
78         numOfVertex, 0));
79     std::vector<int> remaining_degrees(degrees);
80
81     for (int i = 0; i < numOfVertex; ++i) {
82         while (remaining_degrees[i] > 0) {
83             int max_possible_j = numOfVertex - 1;
84             int available = max_possible_j - i;
85
86             if (available <= 0) {
87                 remaining_degrees[i] = 0;
88                 break;
89             }
90
91             int j = i + 1 + (rand() % available);
92
93             if (adjacencyMatrix[i][j] == 0) {
94                 adjacencyMatrix[i][j] = 1;
95                 remaining_degrees[i]--;
96             }
97         }
98     }
99 }

```

```

98
99 void Graph::generateWeightMatrix(double lambda)
100 {
101     static std::random_device rd;
102     static std::mt19937 gen(rd());
103
104     for (int i = 0; i < numOfVertex; i++) {
105         for (int j = 0; j < numOfVertex; j++) {
106             if (adjacencyMatrix[i][j] != 0) {
107                 double U = std::uniform_real_distribution<>(0.0,
108                     1.0)(gen);
109
110                 double random_value = -std::log(1 - U) / lambda;
111
112                 weightMatrix[i][j] = static_cast<int>(
113                     random_value) + 1;
114             }
115         }
116     }
117
118 bool Graph::BellmanFord(int start, int finish, std::vector<int>&
119     distances, std::vector<int>& path, int& iterations)
120 {
121     distances.assign(numOfVertex, INT_MAX);
122     path.assign(numOfVertex, -1);
123     distances[start] = 0;
124     iterations = 0;
125
126     for (int i = 0; i < numOfVertex - 1; ++i) {
127         bool updated = false;
128         for (int u = 0; u < numOfVertex; ++u) {
129             for (int v = 0; v < numOfVertex; ++v) {
130                 if (adjacencyMatrix[u][v] && weightMatrix[u][v]
131                     != 0) { // Если есть ребро u->v
132                     iterations++;
133                     if (distances[u] != INT_MAX && distances[u] +
134                         weightMatrix[u][v] < distances[v]) {
135                         distances[v] = distances[u] +
136                             weightMatrix[u][v];
137                         path[v] = u;
138                         updated = true;
139                     }
140                 }
141             }
142         }
143         if (!updated) break;
144     }
145
146     for (int u = 0; u < numOfVertex; ++u) {
147         for (int v = 0; v < numOfVertex; ++v) {
148             if (adjacencyMatrix[u][v] && weightMatrix[u][v] != 0)

```

```

144         {
145             if (distances[u] != INT_MAX && distances[u] +
146                 weightMatrix[u][v] < distances[v]) {
147                 std::cerr << "Graph contains negative-weight
148                     cycle!" << std::endl;
149                 return false;
150             }
151         }
152     }
153     if (distances[finish] == INT_MAX) {
154         return false;
155     }
156     return true;
}

```

Листинг 2: Исходный код Graph.cpp

```

1  #include "Graph.h"
2
3  const double lambda = 0.2;
4  const double lambda2 = 0.25;
5
6  void printMatrix(std::vector<std::vector<int>> matrix, std::
7      string label)
8  {
9      std::cout << label << ":\n";
10     for (int i = 0; i < matrix.size(); i++) {
11         for (int j = 0; j < matrix.size(); j++) {
12             std::cout << std::setw(3) << matrix[i][j] << std::
13                 setw(3);
14         }
15         std::cout << "\n";
16     }
17     std::cout << "\n";
18 }
19
20 int input()
21 {
22     int value;
23     while (true) {
24         std::cin >> value;
25         if (std::cin.peek() != '\n') {
26             std::cin.clear();
27             std::cin.ignore(std::cin.rdbuf()->in_avail());
28             printf("Input error. Please, enter the integer value
29                 .\n");
30         }
31         else break;
32     }
33     return value;
}

```

```

31 }
32
33 void getStartAndFinish(int& start, int& finish, int numOfVertex)
34 {
35     std::cout << "Enter the number of vertex you want to start
        from (from 1 to " << numOfVertex << "): \n";
36     start = input();
37     while (start<1 || start>numOfVertex)
38     {
39         std::cout << "Input error. Please, enter the value from 0
        to " << numOfVertex << ".\n";
40         start = input();
41     }
42     std::cout << "Enter the number of vertex you want to finish (
        from 1 to " << numOfVertex << "): \n";
43     finish = input();
44     while (finish<1 || finish>numOfVertex)
45     {
46         std::cout << "Input error. Please, enter the value from 1
        to " << numOfVertex << ".\n";
47         finish = input();
48     }
49 }
50
51 void printMenu()
52 {
53     printf("\nEnter the number of option:\n");
54     printf("[0] - Exit\n");
55     printf("[1] - Create a new graph\n");
56     printf("[2] - Bellman - Ford's algorithm\n");
57     printf("\nYour choice: ");
58 }
59
60 int main()
61 {
62     bool flag = true;
63     while (flag) {
64         int vertex;
65         std::cout << "Enter the number of vertices: ";
66         vertex = input();
67         while (vertex < 2 || vertex>100) {
68             std::cout << "Please, enter the integer from 2 to
        100.\n";
69             vertex = input();
70         }
71
72         int edges = vertex - 1;
73         Graph graph(vertex);
74         graph.generateDegrees(lambda);
75
76         for (int i = 0; i < graph.getNumOfVertex(); i++) {
77             std::cout << "Degree of the vertex num." << i + 1 <<

```

```

78         ": " << graph.getDegrees()[i] << "\n";
79     }
80     graph.createAdjMatrix();
81     graph.generateWeightMatrix(lambda2);
82
83     printMatrix(graph.getWeightMatrix(), "\nWeight matrix");
84
85     printMenu();
86     int in;
87     bool anotherFlag = true;
88     while (anotherFlag) {
89         in = input();
90         switch (in) {
91             case 0:
92             {
93                 flag = false;
94                 anotherFlag = false;
95                 break;
96             }
97             case 1:
98             {
99                 std::cout << "You chose [1] - Create a new graph\
100                     n";
101                 anotherFlag = false;
102                 break;
103             }
104             case 2:
105             {
106                 std::cout << "You chose [2] - Bellman - Ford's
107                     algorithm\n";
108                 int start, finish, iterations;
109                 std::vector<int> distances, path;
110                 getStartAndFinish(start, finish, graph.
111                     getNumOfVertex());
112                 start--; finish--;
113                 if (graph.BellmanFord(start, finish, distances,
114                     path, iterations)) {
115                     std::cout << "Shortest distance from " <<
116                         start + 1 << " to " << finish + 1 << ": "
117                         << distances[finish] << "\n";
118
119                     std::vector<std::pair<int, int>> edges;
120                     for (int v = finish; v != start; v = path[v])
121                     {
122                         if (path[v] == -1) break;
123                         edges.push_back(std::make_pair(path[v], v
124                             ));
125                     }
126                     std::reverse(edges.begin(), edges.end());
127
128                     std::cout << "Path edges:\n";

```

```

121         for (const auto& edge : edges) {
122             int u = edge.first;
123             int v = edge.second;
124             std::cout << "( " << u + 1 << " -> " << v
                        + 1 << " ) weight: " << graph.
                        getWeightMatrix()[u][v] << "\n";
125         }
126     }
127     else {
128         std::cout << "No path exists between " <<
            start + 1 << " and " << finish + 1 << ".\n
            ";
129     }
130     printMenu();
131     break;
132 }
133 default:
134 {
135     std::cout << "Please, enter the integer number
            from 0 to 3.\n";
136     printMenu();
137     break;
138 }
139 }
140 }
141 }
142 }

```

Листинг 3: Исходный код main.cpp

4 Описание проведенной инспекции кода

Состав инспекционной группы:

- программист, проектировщик программы – Радионова Полина;
- специалист по тестированию – Черепнов Максим;
- секретарь – Коновалов Даниил.

В начале заседания мной была описана логика программы, представлена ее спецификация и ограничения. В процессе инспекции были заданы 40 вопросов с целью выявления несоответствий спецификации.

Я, как программист и проектировщик программы ответила на вопросы и получила обратную связь от специалиста по тестированию. Секретарем был составлен протокол заседания с записанными вопросами и ответами на них, а также рекомендации по исправлению несоответствий.

4.1 Протокол заседания

1. На каком языке написана программа?

Ответ: на языке C++.

2. Существуют ли в коде обращения к неинициализированным переменным или переменным, которым не были присвоены значения?

Ответ: нет, все переменные инициализируются перед обращением к ним.

3. Используются ли в коде нецелочисленные индексы?

Ответ: нет, все индексы массивов и векторов имеют тип `int`.

4. Есть ли “висячие” ссылки?

Ответ: нет.

5. Сопласуются ли описания структуры данных в разных процедурах?

6. Существуют ли ошибки завышения или занижения значения индекса на единицу при индексации массивов?

Ответ: нет, индексация корректна.

7. Соблюдены ли правила наследования объектов?

Ответ: наследование в коде не используется.

8. Вычисляются ли адреса битовых строк? Передаются ли битовые строки в качестве аргументов?

Ответ: нет, битовые строки не используются.

9. Согласуются ли атрибуты структуры данных и физической записи?

Ответ: данные хранятся в оперативной памяти, физическая запись не применяется.

10. Участвуют ли в вычислениях неарифметические переменные?

Ответ: нет, в вычислениях участвуют только числовые типы.

11. Не выходят ли значения переменных за логически обоснованные пределы?

Ответ: нет, значения ограничены проверками.

12. Выполняются ли вычисления с использованием данных разного типа?

Ответ: да, происходит преобразование `double` в `int` при генерации степеней и весов графа.

13. Возможно ли в коде деление на нуль?

Ответ: нет, делитель защищен проверкой.

14. Правильно ли используются результаты деления целых чисел?

Ответ: да, явное целочисленное деление не используется в критической логике.

15. Критичны ли погрешности, возникающие из-за использования двоичных представлений чисел при вычислениях?

Ответ: нет.

16. Согласуется ли инициализация с классом памяти?

Ответ: да.

17. Имеются ли переменные с похожими именами?

Ответ: да, `lambda` и `lambda2`, `degrees` и `remaining_degrees`.

18. Понятны ли последствия использования атрибутов по умолчанию?

Ответ: да.

19. Правильно ли инициализированы массивы и строки?

Ответ: да, векторы инициализируются нулями через конструктор или `resize`.

20. Есть ли попытки сравнения несопоставимых переменных?

Ответ: нет, сравниваются переменные сопоставимых типов.

21. Корректно ли используются операторы сравнения?

Ответ: да.

22. Корректно ли используются результаты сравнения?

Ответ: да, результаты используются в условиях `if` и циклов.

23. Может ли значение индекса в переключателе превысить число переходов?

Ответ: нет, предусмотрена ветка `default`.

24. Будет ли завершен каждый цикл?

Ответ: да, все циклы имеют условия завершения или фиксированное число повторений.

25. Будет ли завершена программа?

Ответ: да, предусмотрен выход через меню.

26. Есть ли цикл, который может не выполниться из-за входных условий?

Ответ: нет, входные данные валидируются перед входждением.

27. Корректны ли возможные погружения в цикл?

Ответ: да, вложенные циклы логически верны.

28. Имеются ли ошибки диапазона, приводящие к отклонению количества итераций от нужного значения?

Ответ: нет, диапазоны операций соответствуют алгоритмам.

29. Соответствует ли каждому оператору `DO` свой оператор `END`?

Ответ: да, все блоки кода закрыты фигурными скобками корректно.

30. Существуют ли точки ветвления, в которых не учтены все возможные условия?

Ответ: нет, логика ветвления покрывает все сценарии.

31. Корректны ли инструкции `OPEN`?

Ответ: инструкции работы с файлами в коде отсутствуют.

32. Обрабатываются ли ошибки ввода вывода?

Ответ: да, присутствует базовая проверка состояния `cin` и валидация ввода.

33. Присутствуют ли в выходной информации орфографические или грамматические ошибки?

Ответ: нет.

34. Передаются ли константы в качестве аргументов?

Ответ: да, `lambda` и `lambda2` передаются в методы графа.

35. Согласуются ли определения глобальных переменных в разных модулях?

Ответ: да, глобальные константы определены в `main` не конфликтуют.

36. Совпадает ли число формальных параметров с числом аргументов?

Ответ: да.

37. Делаются ли в подпрограмме попытки изменить входные аргументы?

Ответ: да, векторы `distances` и `path` изменяются намеренно (переданы по ссылке).

38. Осуществляется ли проверка корректности входных данных?

Ответ: да, реализована функция `input()` и проверки диапазонов вершин.

39. Выдаются ли какие-нибудь предупреждения или информационные сообщения при компиляции?

Ответ: нет, предупреждения не выдаются.

40. Нет ли пропущенных функций?

Ответ: нет.

4.2 Полученные рекомендации

В результате инспекции кода были получены следующие рекомендации от специалиста по тестированию.

1. **Лишние подключения заголовков в .h файле:** это замедляет компиляцию. Следует подключать только то, что действительно нужно.
2. **Возврат геттеров по константной ссылке:** возврат по значению (`const std::vector<int>`) создает копию всего вектора степеней, что может быть дорого для больших графов. Лучше возвращать по ссылке.
3. **Использование констант:** значения, которые не изменяются (например, минимальное и максимальное количество вершин в графе) лучше хранить в качестве констант.

4. **Построение графа:** методы `generateDegrees` и `createAdjMatrix` используют эвристики и не гарантируют соответствие графа заявленным характеристикам. Лучше использовать для генерации графа конкретные алгоритмы (например, метод Хавела-Хакими).
5. **Улучшение именования:** имена переменных и функций должны понятно отражать их назначение и не перекликаться с названиями других переменных и функций.

4.3 Исправленный код

В листингах 4, 5, 6 приведен исправленный код программы. Были внесены следующие исправления.

1. Часто используемые неизменяемые значения были вынесены в качестве глобальных или объявленных внутри класса константных значений.
2. Методы-геттеры в классе `Graph` теперь возвращают значение по константной ссылке, не создавая копии векторов.
3. Добавлен метод `isGraphicSequence`, проверяющий графичность последовательности методом Хавела-Хакими.
4. Методы `generateDegrees` и `createAdjMatrix` изменены с применением метода `isGraphicSequence`. Теперь создаваемый программой граф гарантированно соответствует предъявляемым к графу требованиям и обрабатывает возможные ошибки.
5. Излишние подключение заголовочных файлов исключены.

```
1  #include <iostream>
2  #include <vector>
3  #include <random>
4  #include <algorithm>
5  #include <limits>
6  #include <iomanip>
7
8  class Graph
9  {
10 private:
11     static constexpr int MAX_VERTEX_DEGREE = 100;
12     static constexpr double MIN_RANDOM_VALUE = 1e-10;
13     static constexpr int MAX_GENERATION_ATTEMPTS = 1000;
14     static constexpr int INFINITY_DISTANCE = INT_MAX;
15
16     int numOfVertex;
17     std::vector<int> degrees;
```

```

18     std::vector<std::vector<int>> adjacencyMatrix;
19     std::vector<std::vector<int>> weightMatrix;
20
21     double randomDouble() {
22         static std::random_device rd;
23         static std::mt19937 gen(rd());
24         static std::uniform_real_distribution<> dis(0.0, 1.0);
25         return dis(gen);
26     }
27
28     bool isGraphicSequence(std::vector<int> degs);
29
30 public:
31     Graph() : numOfVertex(0) {}
32     Graph(int vertex);
33
34     void generateDegrees(double lambda);
35     void createAdjMatrix();
36     void generateWeightMatrix(double lambda);
37
38     bool BellmanFordAlgorithm(int start, int finish, std::vector<
        int>& distances, std::vector<int>& path, int& iterations);
39
40     int getNumOfVertex() const;
41     const std::vector<int>& getDegrees();
42     const std::vector<std::vector<int>>& getAdjMatrix() const;
43     const std::vector<std::vector<int>>& getWeightMatrix() const;
44 };

```

Листинг 4: Исходный код Graph.h

```

1  #include "Graph.h"
2  #include <algorithm>
3  #include <functional>
4  #include <numeric>
5  #include <cmath>
6
7  Graph::Graph(int vertex) : numOfVertex(vertex)
8  {
9      adjacencyMatrix.resize(numOfVertex, std::vector<int>(
        numOfVertex, 0));
10     weightMatrix.resize(numOfVertex, std::vector<int>(numOfVertex
        , 0));
11     degrees.resize(numOfVertex, 0);
12 }
13
14 int Graph::getNumOfVertex() const { return numOfVertex; }
15
16 const std::vector<int>& Graph::getDegrees() { return degrees; }
17
18 const std::vector<std::vector<int>>& Graph::getAdjMatrix() const
    { return adjacencyMatrix; }

```

```

19
20 const std::vector<std::vector<int>>& Graph::getWeightMatrix()
    const { return weightMatrix; }
21
22 bool Graph::isGraphicSequence(std::vector<int> degs)
23 {
24     while (true) {
25         std::sort(degs.begin(), degs.end(), std::greater<int>());
26
27         while (!degs.empty() && degs.back() == 0) {
28             degs.pop_back();
29         }
30
31         if (degs.empty()) {
32             return true;
33         }
34
35         int d = degs.front();
36         degs.erase(degs.begin());
37
38         if (d > degs.size()) {
39             return false;
40         }
41
42         for (int i = 0; i < d; ++i) {
43             degs[i]--;
44             if (degs[i] < 0) {
45                 return false;
46             }
47         }
48     }
49 }
50
51 void Graph::generateDegrees(double lambda)
52 {
53     degrees.clear();
54     degrees.resize(numOfVertex, 0);
55
56     for (int attempt = 0; attempt < MAX_GENERATION_ATTEMPTS; ++
        attempt) {
57         for (int i = 0; i < numOfVertex; ++i) {
58             double r_i = randomDouble();
59             r_i = std::max(r_i, MIN_RANDOM_VALUE);
60             double x_i = -log(r_i) / lambda;
61             degrees[i] = std::min(static_cast<int>(round(x_i)),
                MAX_VERTEX_DEGREE);
62         }
63
64         std::sort(degrees.begin(), degrees.end(), std::greater<
            int>());
65
66         int sum = std::accumulate(degrees.begin(), degrees.end(),

```

```

    0);
67     if (sum % 2 != 0) {
68         continue;
69     }
70
71     if (isGraphicSequence(degrees)) {
72         std::cout << "Graphic sequence generated after " <<
73             attempt + 1 << " attempts.\n";
74         return;
75     }
76
77     std::cout << "Using fallback: generating a path graph.\n";
78     degrees.assign(numOfVertex, 2);
79     degrees[0] = 1;
80     degrees[numOfVertex - 1] = 1;
81
82     if (numOfVertex == 2) {
83         degrees[0] = 1;
84         degrees[1] = 1;
85     }
86 }
87
88 void Graph::createAdjMatrix()
89 {
90     adjacencyMatrix.assign(numOfVertex, std::vector<int>(
91         numOfVertex, 0));
92
93     std::vector<std::pair<int, int>> vertices;
94     for (int i = 0; i < numOfVertex; ++i) {
95         vertices.push_back({ degrees[i], i });
96     }
97
98     while (true) {
99         std::sort(vertices.begin(), vertices.end(),
100             [](const std::pair<int, int>& a, const std::pair<int,
101                 int>& b) {
102                 return a.first > b.first;
103             });
104
105         while (!vertices.empty() && vertices.back().first == 0) {
106             vertices.pop_back();
107         }
108
109         if (vertices.empty()) {
110             break;
111         }
112
113         int d = vertices[0].first;
114         int u_index = vertices[0].second;
115
116         vertices.erase(vertices.begin());

```

```

115
116     if (d > vertices.size()) {
117         std::cerr << "Error in Havel-Hakimi algorithm:
118             insufficient vertices.\n";
119         break;
120     }
121
122     for (int i = 0; i < d; ++i) {
123         vertices[i].first--;
124         int v_index = vertices[i].second;
125         adjacencyMatrix[u_index][v_index] = 1;
126     }
127
128     for (int i = 0; i < numOfVertex; ++i) {
129         for (int j = i + 1; j < numOfVertex; ++j) {
130             if (adjacencyMatrix[i][j] == 1) {
131                 adjacencyMatrix[j][i] = 1;
132             }
133         }
134     }
135 }
136
137 void Graph::generateWeightMatrix(double lambda)
138 {
139     static std::random_device rd;
140     static std::mt19937 gen(rd());
141
142     for (int i = 0; i < numOfVertex; i++) {
143         for (int j = 0; j < numOfVertex; j++) {
144             if (adjacencyMatrix[i][j] != 0) {
145                 double U = std::uniform_real_distribution<>(0.0,
146                     1.0)(gen);
147                 double random_value = -std::log(1 - U) / lambda;
148                 weightMatrix[i][j] = static_cast<int>(
149                     random_value) + 1; // +1 чтобы вес был миниму
150                     м 1
151             }
152         }
153     }
154 }
155
156 bool Graph::BellmanFordAlgorithm(int start, int finish, std:::
157     vector<int>& distances, std:::vector<int>& path, int&
158     iterations)
159 {
160     distances.assign(numOfVertex, INFINITY_DISTANCE);
161     path.assign(numOfVertex, -1);
162     distances[start] = 0;
163     iterations = 0;
164
165     for (int i = 0; i < numOfVertex - 1; ++i) {

```



```

161     bool updated = false;
162     for (int u = 0; u < numOfVertex; ++u) {
163         for (int v = 0; v < numOfVertex; ++v) {
164             if (adjacencyMatrix[u][v] && weightMatrix[u][v]
165                 != 0) {
166                 iterations++;
167                 if (distances[u] != INFINITY_DISTANCE &&
168                     distances[u] + weightMatrix[u][v] <
169                         distances[v]) {
170                     distances[v] = distances[u] +
171                         weightMatrix[u][v];
172                     path[v] = u;
173                     updated = true;
174                 }
175             }
176         }
177         if (!updated) break;
178     }
179     for (int u = 0; u < numOfVertex; ++u) {
180         for (int v = 0; v < numOfVertex; ++v) {
181             if (adjacencyMatrix[u][v] && weightMatrix[u][v] != 0)
182             {
183                 if (distances[u] != INFINITY_DISTANCE &&
184                     distances[u] + weightMatrix[u][v] < distances
185                         [v]) {
186                     std::cerr << "Graph contains negative-weight
187                         cycle!" << std::endl;
188                     return false;
189                 }
190             }
191         }
192     }
193     return distances[finish] != INFINITY_DISTANCE;
194 }

```

Листинг 5: Исходный код Graph.cpp

```

1  #define _CRT_SECURE_NO_WARNINGS
2  #include "Graph.h"
3
4  const double LAMBDA_DEGREES = 0.2;
5  const double LAMBDA_WEIGHTS = 0.25;
6  const unsigned int RANDOM_SEED = 42;
7  const int MIN_VERTICES = 2;
8  const int MAX_VERTICES = 100;
9  const int MENU_EXIT = 0;
10 const int MENU_NEW_GRAPH = 1;
11 const int MENU_BELLMAN_FORD = 2;
12

```

```

13 void printMatrix(std::vector<std::vector<int>> matrix, std::
    string label)
14 {
15     std::cout << label << ":\n";
16     for (int i = 0; i < matrix.size(); i++) {
17         for (int j = 0; j < matrix.size(); j++) {
18             std::cout << std::setw(3) << matrix[i][j] << std::
                setw(3);
19         }
20         std::cout << "\n";
21     }
22     std::cout << "\n";
23 }
24
25 int validateInput()
26 {
27     int value;
28     while (true) {
29         std::cin >> value;
30         if (std::cin.peek() != '\n') {
31             std::cin.clear();
32             std::cin.ignore(std::cin.rdbuf()->in_avail());
33             printf("Input error. Please, enter the integer value
                .\n");
34         }
35         else break;
36     }
37     return value;
38 }
39
40 void getStartAndFinish(int& start, int& finish, int numOfVertex)
41 {
42     std::cout << "Enter the number of vertex you want to start
        from (from 1 to " << numOfVertex << "): \n";
43     start = validateInput();
44     while (start < 1 || start > numOfVertex)
45     {
46         std::cout << "Input error. Please, enter the value from 1
            to " << numOfVertex << ":\n";
47         start = validateInput();
48     }
49     std::cout << "Enter the number of vertex you want to finish (
        from 1 to " << numOfVertex << "): \n";
50     finish = validateInput();
51     while (finish < 1 || finish > numOfVertex)
52     {
53         std::cout << "Input error. Please, enter the value from 1
            to " << numOfVertex << ":\n";
54         finish = validateInput();
55     }
56 }
57

```

```

58 void printMenu()
59 {
60     printf("\nEnter the number of option:\n");
61     printf("[%d] - Exit\n", MENU_EXIT);
62     printf("[%d] - Create a new graph\n", MENU_NEW_GRAPH);
63     printf("[%d] - Bellman - Ford's algorithm\n",
64         MENU_BELLMAN_FORD);
65     printf("\nYour choice: ");
66 }
67
68 int main()
69 {
70     bool flag = true;
71     while (flag) {
72         int vertex;
73         std::cout << "Enter the number of vertices: ";
74         vertex = validateInput();
75         while (vertex < MIN_VERTICES || vertex > MAX_VERTICES) {
76             std::cout << "Please, enter the integer from " <<
77                 MIN_VERTICES << " to " << MAX_VERTICES << ".\n";
78             vertex = validateInput();
79         }
80
81         Graph graph(vertex);
82         graph.generateDegrees(LAMBDA_DEGREES);
83
84         for (int i = 0; i < graph.getNumOfVertex(); i++) {
85             std::cout << "Degree of the vertex num." << i + 1 <<
86                 ": " << graph.getDegrees()[i] << "\n";
87         }
88         graph.createAdjMatrix();
89
90         graph.generateWeightMatrix(LAMBDA_WEIGHTS);
91
92         printMatrix(graph.getWeightMatrix(), "\nWeight matrix");
93
94         printMenu();
95         int in;
96         bool anotherFlag = true;
97         while (anotherFlag) {
98             in = validateInput();
99             switch (in) {
100                 case MENU_EXIT:
101                     {
102                         flag = false;
103                         anotherFlag = false;
104                         break;
105                     }
106                 case MENU_NEW_GRAPH:
107                     {
108                         std::cout << "You chose [" << MENU_NEW_GRAPH << "
109                             ] - Create a new graph\n";
110                     }
111             }
112         }
113     }
114 }

```

```

106         anotherFlag = false;
107         break;
108     }
109     case MENU_BELLMAN_FORD:
110     {
111         std::cout << "You chose [" << MENU_BELLMAN_FORD
112             << "]" - Bellman - Ford's algorithm\n";
113         int start, finish, iterations;
114         std::vector<int> distances, path;
115         getStartAndFinish(start, finish, graph.
116             getNumOfVertex());
117         start--;
118         finish--;
119
120         if (graph.BellmanFordAlgorithm(start, finish,
121             distances, path, iterations)) {
122             std::cout << "Shortest distance from " <<
123                 start + 1 << " to " << finish + 1 << ": "
124                 << distances[finish] << "\n";
125             std::cout << "Number of iterations: " <<
126                 iterations << "\n";
127
128             std::vector<std::pair<int, int>> edges;
129             for (int v = finish; v != start; v = path[v])
130             {
131                 if (path[v] == -1) break;
132                 edges.push_back(std::make_pair(path[v], v
133                     ));
134             }
135             std::reverse(edges.begin(), edges.end());
136
137             std::cout << "Path edges:\n";
138             for (const auto& edge : edges) {
139                 int u = edge.first;
140                 int v = edge.second;
141                 std::cout << "(" << u + 1 << " -> " << v
142                     + 1 << " ) weight: " << graph.
143                     getWeightMatrix()[u][v] << "\n";
144             }
145         }
146         else {
147             std::cout << "No path exists between " <<
148                 start + 1 << " and " << finish + 1 << ".\n
149                 ";
150         }
151         printMenu();
152         break;
153     }
154     default:
155     {
156         std::cout << "Please, enter the integer number
157             from " << MENU_EXIT << " to " <<

```

```
145         MENU_BELLMAN_FORD << ".\n";
146     printMenu();
147     break;
148 }
149 }
150 }
151 return 0;
152 }
```

Листинг 6: Исходный код main.cpp

Заключение

В ходе выполнения лабораторной работы были изучены методы ручного тестирования – инспекция кода, сквозной просмотр, проверка за столом и рецензирования. Методом инспекции кода было проведено тестирование подготовленной заранее программы. В проведении инспекционного собрания участвовали: программист, секретарь и специалист по тестированию. В результате собрания секретарем был составлен протокол инспекции, специалист по тестированию дал мне рекомендации по улучшению программы. Мною были проанализированы данные рекомендации и внесены исправления в код.

В результате выполненной работы код был улучшен: граф гарантированно генерируется корректно, а также улучшилась читаемость кода. Я сделала вывод, что ручные методы тестирования, в частности инспекция кода, могут быть эффективны, особенно для небольших программ.